ChatGPT

OpenVSP Skinning 參數與 Python API 控制方法

Skinning 參數概念

OpenVSP 的 **Skinning** 用於在不同跨截面（X‑Section）之間創建平滑的外形，方式是為每個截面定義前（Fore）/後（Aft）兩側的切線向量。每個側面有四個控制量，可以獨立或對稱調整，並可選擇前後等值（「=」按鈕）或使用對稱（Symmetry）設定 [1]：

參數	定義與作用
Angle (切線角)	決定截面上切線向量「伸出」或「收縮」的方向。Fore/Aft 兩側角度符號相反：例如後側向外膨脹為正角度，前側則以負角度表達相同效果 [2]。角度主要決定表面在截面處的趨勢，典型範圍約在 ±180°。
Slew (扭轉量)	控制切線向量繞截面法線的旋轉，使特徵線在截面平面內旋轉。當截面有俯仰或轉角時，需要調整 slew 以保持相鄰截面特徵線方向一致 [3]。OpenVSP 的改版記錄亦指出 slew 控制曲率方向“在”表面，而 angle 控制曲率方向“離”表面 [4]。
Strength (強度)	決定切線向量的長度（特徵線伸展距離）。較大的 strength 會讓截面之間的過渡更加飽滿；若為 0，特徵線收斂到零容易造成自交或幾何錯誤，因此建議使用正值 [5]。
Curvature (曲率)	直接控制切線曲率，可在少數需要高度控制曲率的情況下使用；通常 angle、slew、strength 就能形成所需外形，曲率用得較少 [6]。

此外，**C0/C1/C2** 連續性用於調整截面之間的光滑程度：C0 只保證位置連續，C1 同時保證切線連續（通常推薦），C2 進一步保證曲率連續 [7]。

Skinning Python API 控制

OpenVSP 提供一組函式控制跨截面的 Skinning 參數，以下為主要函式及使用方式：

- `ResetXSecSkinParms(xsec_id)`：重置指定截面的 angle、slew、strength、curvature 以及對稱/等值設定 [8]。
- `SetXSecContinuity(xsec_id, cx)`：設定跨截面連續性；`cx` 取值 0、1 或 2，分別為 C0、C1、C2 [9]。
- `SetXSecTanAngles(xsec_id, side, top, right, bottom, left)`：設定截面兩側的切線角度（角度以度為單位） [10]。
- `SetXSecTanSlews(xsec_id, side, top, right, bottom, left)`：設定截面兩側的 slew（扭轉量） [11]。
- `SetXSecTanStrengths(xsec_id, side, top, right, bottom, left)`：設定截面兩側的強度 [12]。
- `SetXSecCurvatures(xsec_id, side, top, right, bottom, left)`：設定截面曲率 [13]。

參數 `side` 可用 `vsp.XSEC_FORE_SIDE`、`vsp.XSEC_AFT_SIDE` 或 `vsp.XSEC_BOTH_SIDES` 指定前側、後側或同時設定。`top/right/bottom/left` 用於矩形截面指定四邊，對於圓形或對稱截面可同時用同一值。

範例：設定機身截面的 Skinning

```
# 假設已建立機身並取得其堆疊截面 ID 列表 xsec_ids
import openvsp as vsp

for xs in xsec_ids:
    # 將截面設為 C1 連續
    vsp.SetXSecContinuity(xs, 1)
    # 同時設定前後兩側 angle、slew、strength（單位為度、度、無量綱）
    vsp.SetXSecTanAngles(xs, vsp.XSEC_BOTH_SIDES, 10.0, 0, -10.0, 0)  # 上/右/下/左
    vsp.SetXSecTanSlews(xs, vsp.XSEC_BOTH_SIDES, 5.0, 0, -5.0, 0)
    vsp.SetXSecTanStrengths(xs, vsp.XSEC_BOTH_SIDES, 0.8, 0.8, 0.8, 0.8)
    # 需要特殊曲率時才設定
    vsp.SetXSecCurvatures(xs, vsp.XSEC_BOTH_SIDES, 0.2, 0.2, 0.2, 0.2)
```

調整時可先將 `Set` 切換為 Auto（讓 OpenVSP 自動生成切線），並在必要時逐一開啟 manual；若需要前後對稱，啟用 `=` 鍵即可使 Fore/Aft 參數相等 [1]。

OpenVSP Parasite Drag 阻力分析

分析概念與限制

OpenVSP 的 **Parasite Drag** 工具利用現有模型幾何與半經驗公式估算零升阻力 CD_0。其主要功能包括：

- 自動呼叫 Comp Geom 計算各部件的淨濕面積，並將幾何特徵填入阻力分析 [14]。
- 提供多種大氣模型 (U.S. Standard 1976、USAF 1966) 以及單位選擇，可透過設定任何兩個變數（壓力、密度、溫度）或直接指定 Mach 數與雷諾數來定義流場 [15]。
- 提供多種 laminar/turbulent 摩擦係數方程式與翼/機身形狀因子公式，使用者可為每個部件選擇適當的方程式並設定干擾因子 (Q) 與層流比例 [16]。Laminar 方程式包括 Blasius 等，Turbulent 方程式包括 Spalding、Schoenherr、Power Law 等 [17]；機身形狀因子可選 Hoerner、Torenbeek、Shevell、Jenkinson 等 [18]，翼形狀因子可選 Schemensky 4/6 系列、Hoerner、Torenbeek、Kroo 等 [19]。
- 可設定 Excrescence (附加阻力) 用於考慮天線、接縫、粗糙度、昆蟲等「現實世界」元件，能以固定 drag counts、drag coefficient、幾何阻力百分比或固定 drag area 加入 [20]。

限制：此工具針對固定翼飛行器，不能計算橫流或旋翼洗流；特徵長度來源於幾何本身，不隨攻角改變 [21]。由於採用半經驗公式，結果具有近似性；VSPAERO 的粘性阻力估算通常會低估真實阻力 [22]。因此在高精度需求下，應對比試驗或 CFD 結果並藉由 Excrescence 調校。

Python API 操作流程

以下說明如何利用 Python API 自動執行 Parasite Drag 分析並讀取結果：

1. **更新幾何與預處理**：完成模型設計後，呼叫 `vsp.Update()` 更新幾何；在阻力分析前建議先執行 Comp Geom（`vsp.ExecAnalysis("CompGeom")`），確保濕面積與交點正確。

2. **設定分析**：

```python
import openvsp as vsp

# 指定分析名稱
analysis = "ParasiteDrag"
# 將輸入設定回預設值
vsp.SetAnalysisInputDefaults(analysis)
# 必要時選擇是否重新計算幾何（0=否，1=是）
vsp.SetIntAnalysisInput(analysis, "RecomputeGeom", [0])
# 選擇長度單位：0=英寸、1=英呎、2=米… 視 enumerations 而定
vsp.SetIntAnalysisInput(analysis, "LengthUnit", [2])  # 以公尺為長度單位
# 選擇自由流條件定義方式：例如 5 表示給定 Re/L 與 Mach 數
vsp.SetIntAnalysisInput(analysis, "FreestreamPropChoice", [5])
# 設定自由流參數（必須以列表方式傳入）
vsp.SetDoubleAnalysisInput(analysis, "Mach", [0.3])        # 馬赫數
vsp.SetDoubleAnalysisInput(analysis, "Re_L", [5e6])        # 雷諾數 (基於特徵長度)
# 若使用速度定義，可設定速度單位與速度值
vsp.SetIntAnalysisInput(analysis, "VelocityUnit", [vsp.V_UNIT_M_S])  # 單位：m/s [23]
vsp.SetDoubleAnalysisInput(analysis, "Vinf", [50.0])       # 自由流速度
# 選擇 laminar 摩擦係數方程式
vsp.SetIntAnalysisInput(analysis, "LamCfEqn", [vsp.CF_LAM_BLASIUS])  # Blasius laminar 方程式 [24]
# 選擇 turbulent 摩擦係數方程式
vsp.SetIntAnalysisInput(analysis, "TurbCfEqn", [vsp.CF_TURB_POWER_LAW_BLASIUS])  # Power Law Blasius turbulent 方程式 [25]
# 選擇機身和翼的形狀因子方程式（每個部件可不同）
# 例如對所有 fuselage 設為 Hoerner Streamlined Body
#  需遍歷各機身幾何，使用 vsp.SetParmVal() 將其 ParasiteDragProps->FFEqn 設為 vsp.FF_B_HOERNER_STREAMBODY [26]
# 此處略示範：假設 fuselage_id 為機身 ID
vsp.SetParmVal(fuselage_id, "FFEqn", "ParasiteDragProps", vsp.FF_B_HOERNER_STREAMBODY)
# 同理對機翼使用 Schemensky 4 Series Airfoil 方程式
vsp.SetParmVal(wing_id, "FFEqn", "ParasiteDragProps", vsp.FF_W_SCHEMENSKY_4_SERIES_AF)
# 設定 Excrescence 可使用 AddExcrescence
vsp.AddExcrescence("Miscellaneous", vsp.EXCRESCENCE_COUNT, 8.5)  # 新增 8.5 的 drag count [27]
# 更新 Parasite Drag 工具
vsp.UpdateParasiteDrag()
```

> **注意：** API 的輸入均為向量，即使只有單一值也必須傳入 Python list，如 `[0]` 或 `[50.0]`，否則會出現參數類型錯誤 [28]。

1. **執行分析：**

```python
result_id = vsp.ExecAnalysis(analysis)
# 讀取結果：總阻力係數 (Total_CD_Total) 位於索引 0
cd_tot = vsp.GetDoubleResults(result_id, "Total_CD_Total", 0)[0]
print(f"CD0 = {cd_tot}")
```

```
# 將完整結果輸出成 CSV
vsp.WriteResultsCSVFile(result_id, "parasite_drag_results.csv")
```

1. **檢查與調整**：分析結果與 GUI 可能略有差異。建議確認 API 輸入與 GUI 設定完全一致，包含各部件的 **ParasiteDragProps** 參數（形狀因子、干擾因子、層流比例等）。若需要校正，可透過 Excrescence 增加固定阻力或百分比調整 [20]。為維持準確度，建議在不同飛行條件或部件組合下選擇合適的摩擦係數和形狀因子方程式 [16]。

誤差與建議

- **半經驗公式的局限**：Parasite Drag 工具採用簡化的摩擦係數與形狀因子公式，未考慮橫流或旋翼洗流，因此對複雜構型（如高攻角、VTOL 旋翼機等）可能誤差較大 [21]。
- **低估阻力**：VSPAERO 僅估算誘導阻力，粘性阻力估算會低估實際值；Google 群組討論指出 VSPAERO 的 CD_0 估計通常較樂觀 [22]。
- **精度提升方法**：
- 根據機身/翼型特性選擇合適方程式並調整層流比例、干擾因子。 [16]
- 使用 Excrescence 加入接縫、裝備與表面粗糙度的額外阻力 [20]。
- 將 API 結果與風洞試驗或 CFD 檢驗，並通過調整形狀因子或 Excrescence 進行校正。
- 若需要考慮黏性流場，建議配合高階 CFD 工具或使用 VSPAERO 的面板方法並添加外部摩擦阻力估算。

以上內容總結了 OpenVSP 中 skinning 參數的物理意義與 Python API 操作方法，並說明如何使用 Parasite Drag 工具進行零升阻力計算、可調整的方程式與參數，以及使用 API 的範例流程與注意事項。希望能幫助您在自動化設計與阻力估算中取得較好的準確度與效率。

---

[1] [2] [3] [5] [6] [7] OpenVSPにおける胴体のモデリング | mtk_birdman's blog
https://mtkbirdman.com/openvsp-fuselage-modeling

[4] raw.githubusercontent.com
https://raw.githubusercontent.com/OpenVSP/OpenVSP/main/CHANGELOG.md

[8] [9] [10] [11] [12] [13] OpenVSP API Documentation: XSec and Airfoil Functions
https://openvsp.org/api_docs/latest/group___x_sec.html

[14] [15] [16] [20] [21] OpenVSP Parasite Drag Tool - NASA
https://www.nasa.gov/reference/openvsp-parasite-drag-tool/

[17] [18] [19] [23] [24] [25] [26] OpenVSPAPI: Enumerations
https://openvsp.org/api_docs/3.21.0/group___enumerations.html

[22] About accuracy of drag/lift-to-drag ratio
https://groups.google.com/g/openvsp/c/lafyJgeT7Uw

[27] OpenVSP API Documentation: Parasite Drag Functions
https://openvsp.org/api_docs/latest/group___parasite_drag.html

[28] Parasite Drag (openVSP Python API)
https://groups.google.com/g/openvsp/c/SOQ1xCApdmc